

Web Typography & Text Elements

A build checklist for the Primitiv Figma library. Every web content element you need, what it's made of, which existing tokens it binds to, and which need a new namespace. Drag this into Figma as a reference board.

Generated from `packages/tokens/src/*.json` (the repo copy of the live Figma variables). Values shown are the resolved primitives at the time of generation. Branch `claude/figma-typography-planning-cbohph`.

COMPONENT

stateful / composed

TEXT STYLE

exists, just confirm colour

INLINE MARK

character style /

span

EXISTS

NEEDS BUILDING

NEW TOKEN NAMESPACE

Contents

1. The model — two layers, one rule

2. Reference: the type scale (per density)

3. Reference: semantic colour (light / dark)

4. Block prose — text styles

5. Structured content — components
6. Inline / text-level — marks

7. New token namespaces to create

8. The monospace gap

9. Master checklist (one-glance)

1 · The model — two layers, one rule

Typography in Primitiv is already solved at the **variable** level. Building web elements is mostly **composition + colour**, not inventing type scales.

Layer	What it does	You touch it when...
Context collection 4 modes: Dense · Compact · Comfortable · Spacious	Holds every size decision: <code>heading/*</code> , <code>body/*</code> , <code>label/*</code> , <code>display/*</code> , <code>overline/*</code> , plus all spacing/anatomy. The active value is chosen by the frame's density mode override .	Binding font size / line-height / spacing. Adding a new structural namespace (lists, tables...).
Intent collection 2 modes: Light · Dark	Holds every colour decision: <code>content/*</code> , <code>surface/*</code> , <code>border/*</code> , <code>action/*</code> . Light/Dark inversion is automatic via the palette's own modes.	Binding any fill, stroke, or text colour.

The two-collection rule

Context for size & spacing. Intent (content / surface / border / action) for colour. Bind to those and both density switching and light/dark theming work for free via frame mode overrides — no rebinding, no dark values to author.

The one gotcha that breaks everything — text styles silently kill density. A Figma `TextStyle` has no `setExplicitVariableModeForCollection`, so it always resolves at the collection's **default mode (Compact)** regardless of the frame's density override.

- **Loose document text** (a heading dropped into a density-overridden page frame) → a text style is fine; it picks up the frame's density.
- **Inside a reusable component** → **never apply a text style**. Bind `font-family`, `font-style`, `font-size`, `line-height` **inline** via `node.setBoundVariable`. If any of the four is missing from `boundVariables`, that node is unbound and won't respond to density.

Density is frame-owned. Never set an explicit Context mode override on a component *variant* — it locks instances to one density and breaks the consumer's frame-level switching.

Font families & weights (the whole set)

Token	Family	Weights available	Role
<code>font-family.heading</code>	Khand (condensed)	Regular 400 · Medium 500 · SemiBold 600	Headings, labels, overline, display, button text
<code>font-family.text</code>	Asta Sans	Regular 400 · Bold 700 (weight primitive exists; see note)	Body, descriptive copy, sub-labels, input value text

Primitives define weights up to `font-weight.bold` (700), but the `font-style` string tokens cover only Regular / Medium / SemiBold — add `font-style.bold` = "Bold" to bind body bold cleanly. No monospace face exists — see §8. Never use Inter; it is not in the system.

2 · Reference — the type scale per density

Each cell is **font-size / line-height** in px. The role's family/weight is constant across densities. These are the variables your elements bind to; the column is selected by the frame's Context mode.

HEADING/H1–H6 · KHAND SEMIBOLD

Slot	Dense	Compact (default)	Comfortable	Spacious
heading/h1	18 / 24	32 / 40	48 / 56	88 / 96
heading/h2	17 / 20	28 / 36	40 / 48	68 / 76
heading/h3	16 / 20	26 / 36	32 / 40	52 / 60
heading/h4	15 / 20	22 / 32	28 / 36	36 / 44
heading/h5	14 / 16	20 / 28	24 / 32	28 / 40
heading/h6	13 / 16	18 / 28	20 / 28	20 / 32

BODY/XS–XL · ASTA SANS REGULAR

Slot	Dense	Compact	Comfortable	Spacious
body/xs	10 / 12	12 / 16	12 / 16	12 / 20
body/sm	11 / 14	14 / 20	14 / 20	14 / 24
body/md	12 / 16	16 / 24	16 / 24	16 / 28
body/lg	13 / 16	18 / 28	20 / 32	22 / 36
body/xl	14 / 20	20 / 32	22 / 36	24 / 40

LABEL/XS–XL · KHAND SEMIBOLD (INTERACTIVE-CONTROL LABELS; THE CONTRAST FACE VS BODY)

Slot	Dense	Compact	Comfortable	Spacious
label/xs	10 / 12	12 / 16	13 / 20	14 / 20
label/sm	11 / 14	14 / 20	16 / 24	18 / 28
label/md	12 / 16	16 / 24	18 / 28	20 / 32
label/lg	13 / 16	18 / 28	20 / 32	22 / 36
label/xl	14 / 20	20 / 32	22 / 36	24 / 40

DISPLAY/LG–XL · KHAND MEDIUM OVERLINE · KHAND MEDIUM, UPPERCASE

Slot	Dense	Compact	Comfortable	Spacious
display/lg	20 / 24	36 / 44	56 / 64	120 / 128
display/xl	22 / 28	40 / 48	68 / 76	160 / 168
overline	10 / 12	12 / 16	12 / 16	12 / 20

3 · Reference — semantic colour (light / dark)

Bind text/fill/stroke to these — never to raw `neutral/*`. The same alias serves both modes; the palette's Light/Dark modes do the inversion. Aliases below; resolve hex in Figma.

CONTENT/* — TEXT & ICON COLOUR

Token	Light alias	Dark alias	Use for
<code>content/primary</code>	<code>neutral/900</code>	<code>neutral/50</code>	Headings, body, default text
<code>content/secondary</code>	<code>neutral/700</code>	<code>neutral/200</code>	Lead text, list markers, captions-with-weight
<code>content/muted</code>	<code>neutral/500</code>	<code>neutral/400</code>	Captions, helper, fine print, <code><small></code>
<code>content/disabled</code>	<code>neutral/400</code>	<code>neutral/500</code>	Disabled text
<code>content/error</code>	<code>danger/500</code>	<code>danger/500</code>	Error text, required mark
<code>content/inverse</code>	<code>color/white</code>	<code>color/white</code>	Text on a dark/inverse surface
<code>content/on-action</code>	<code>absolute-white</code>	<code>absolute-white</code>	Text on a coloured action fill

SURFACE/* · BORDER/* · ACTION/LINK/*

Token	Light	Dark	Use for
<code>surface/default</code>	<code>absolute-white</code>	(palette)	Page / card background
<code>surface/subtle</code>	<code>neutral/100</code>	<code>neutral/800</code>	Inline code bg, code block bg, zebra rows
<code>surface/raised</code>	<code>neutral/50</code>	(palette)	Table header bg, callout bg
<code>border/subtle</code>	<code>neutral/200</code>	(palette)	Dividers, table cell borders, code outline
<code>border/default</code>	<code>neutral/300</code>	(palette)	Default control borders
<code>border/strong</code>	<code>neutral/500</code>	(palette)	Blockquote accent bar
<code>border/focus</code> · <code>focus/ring</code>	<code>brand/500</code>	<code>brand/500</code>	Focus ring on links/controls
<code>action/link/foreground/default</code>	<code>brand/500</code>	<code>brand/500</code>	Link — resting
<code>action/link/foreground/hover</code>	<code>brand/600</code>	<code>brand/300</code>	Link — hover
<code>action/link/foreground/active</code>	<code>brand/700</code>	<code>brand/200</code>	Link — active/pressed
<code>action/link/foreground/disabled</code>	<code>brand/500 + 50% opacity</code>	<code>brand/500 + 50%</code>	Link — disabled (opacity on the node)

4 · Block prose — text styles TEXT STYLE

These ride entirely on existing variables. No components needed — confirm the colour pairing and (for new roles) add the text style. Drop into density-overridden page frames; they respond to density automatically.

Element	Status	Type variable	Colour	Notes
Display / hero	EXISTS	display/lg-xl	content/primary	Marketing hero only; Khand Medium.
Headings h1–h6	EXISTS	heading/h1–h6	content/primary	Map HTML level → slot 1:1.
Paragraph / body	EXISTS	body/md	content/primary	Default running text.
Lead / intro paragraph	ADD STYLE	body/lg or xl	content/secondary	First paragraph of an article.
Caption / helper / fine print	ADD STYLE	body/xs	content/muted	Below media, form helper.
Overline / eyebrow	EXISTS	overline	content/secondary	Uppercase, +letter-spacing.
<address>	ADD STYLE	body/sm italic	content/secondary	Contact block.

5 · Structured content — components

The real build work. Each carries spacing/border/fill decisions beyond pure type, so each is a component with inline-bound typography. Per-element anatomy below.

Link

COMPONENT

BUILD

The only stateful inline element. An in-flow text link — distinct from the Button "link variant" (which is a button that looks like a link).

PARTS

- **Label** — text node. Inherits the surrounding body size (don't fix a size; bind to the same `body/*` as context, default `body/md`).
- **Underline** — text-decoration (or a 1px bottom border for offset control).
- **Leading / trailing icon** (optional) — e.g. external-link glyph; size to body cap-height.
- **Focus ring** — reuse the framed-control 2-frame ring or an outline on focus.

VARIANTS / PROPERTIES

- **State** (variant): default · hover · active · visited · focus · disabled
- **Tone** (variant): default · muted (`content/secondary`) · inverse (on dark)
- **Icon** (boolean/instance-swap): none · leading · trailing
- **Underline** (boolean): always · hover-only

TOKEN BINDINGS

Part	Property	Token
Label	font (inline)	<code>body/{size}/*</code>
Label / underline	fill	<code>action/link/foreground/{state}</code>
Disabled	opacity	50% on the node (token stays default)
Visited	fill	<code>brand/700</code> (or dedicated <code>action/link/visited</code> if added)
Focus ring	stroke	<code>focus/ring</code>

Visited has no token today. Either reuse `active`'s `brand/700` or add `action/link/foreground/visited` . Decide before building.

List + List Item

COMPONENT

BUILD

LIST/*

Unordered & ordered. Build as two components: a **List** container (vertical auto-layout) holding **List Item** instances.

PARTS

- **List** — vertical auto-layout; item gap = `list/item-gap` ; left padding = `list/indent` .
- **List Item** — horizontal auto-layout: Marker + Content.
- **Marker** — bullet glyph (disc/circle/square) or number/letter text; fill `content/secondary` .
- **Content** — body text; may itself contain a nested List (recursion → indent compounds).

VARIANTS / PROPERTIES

- **Type**: unordered · ordered
- **Marker style**: disc · circle · square · decimal · lower-alpha · lower-roman
- **Item**: leaf · has-nested · with-icon-marker (e.g. check for task lists)

TOKEN BINDINGS

Part	Property	Token
Content	font (inline)	<code>body/md/*</code>
Content	fill	<code>content/primary</code>
Marker	fill	<code>content/secondary</code>
List	item gap	<code>list/item-gap</code> (new)
List	indent (padding-left)	<code>list/indent</code> (new)
List Item	marker – content gap	<code>list/marker-gap</code> (new)

Description List (dl / dt / dd)

COMPONENT

BUILD

Easy to forget, very common (glossaries, key/value, metadata).

PARTS

- **Pair** — term + definition group.
- **Term (dt)** — `label/sm` (Khand SemiBold) or body semibold; `content/primary` .
- **Definition (dd)** — `body/md` ; `content/secondary` .

VARIANTS

- **Layout**: stacked (term above) · inline two-column (term left, definition right)

BINDINGS

Part	Token
Term font	<code>label/sm/*</code> (face contrast does the hierarchy)
Term fill	<code>content/primary</code>
Definition font	<code>body/md/*</code>
Definition fill	<code>content/secondary</code>
Pair gap / row gap	<code>space-4</code> / <code>space-12</code> (or <code>list/*</code>)

Blockquote

COMPONENT

BUILD

QUOTE/*

PARTS

- **Container** — left accent bar + inline padding.
- **Quote text** — `body/lg` , italic, `content/secondary` .
- **Citation / footer** (optional) — `body/sm` , `content/muted` , "— Name".

VARIANTS

- **Citation**: with · without
- **Tone**: default (neutral bar) · accent (brand bar)

BINDINGS

Part	Property	Token
Accent bar	stroke / fill	<code>border/strong</code> (default) · <code>border/focus</code> =brand (accent)
Bar width	border-width	<code>border-width/2</code> or <code>/3</code>
Container	padding-inline	<code>quote/padding-inline</code> (new) → <code>space-16/20/24</code>
Quote text	font / fill	<code>body/lg/*</code> italic · <code>content/secondary</code>
Citation	font / fill	<code>body/sm/*</code> · <code>content/muted</code>

Confirm whether Asta Sans ships an italic style. If it doesn't, skip italic and lean on the accent bar + indentation for the quote signal. Decide and document.

Pull quote

COMPONENT

BUILD

A large decorative quote, not the same as blockquote. Often centred, no attribution, big type.

Part	Token
Quote text	<code>heading/h4</code> or <code>display/lg</code> · <code>content/primary</code>
Quotation marks	decorative, <code>content/muted</code> at large size
Block margin	<code>space-32</code> / <code>space-48</code>

Inline code

<code>

COMPONENT

BUILD

MONO?

PARTS

- **Span** — text with a tinted box: bg + inline padding + small radius + subtle border.

Property	Token
Font	monospace — does not exist yet (§8)
Size	~0.9em of surrounding body
Text fill	content/primary
Background	surface/subtle
Border	border/subtle (1px, optional)
Padding-inline	space-4
Radius	radii/4

Code block

<pre>

COMPONENT

BUILD

CODE/* · MONO

PARTS

- **Container** — surface fill, radius, padding, horizontal scroll.
- **Header** (optional) — filename label + Copy button (instance-swap the Button).
- **Line numbers** (optional) — gutter, content/muted .
- **Code text** — monospace; syntax colours out of scope (use a token set later).

| Part | Property | Token |
|-----------|----------|---|
| Container | fill | surface/subtle (or surface/inverse for dark code) |
| Container | border | border/subtle |
| Container | radius | radii/8 |
| Container | padding | code/padding (new) → space-12/16 |
| Code text | font | monospace (§8) · fill content/primary |
| Gutter | fill | content/muted |

Table

COMPONENT

BUILD

TABLE/*

The heavyweight. Build from atomic Cell components composed into Rows, composed into a Table.

PARTS

- **Header cell (th)** — label/sm or body/sm semibold, content/secondary, on surface/raised.
- **Body cell (td)** — body/sm, content/primary.
- **Row** — horizontal auto-layout of cells; bottom border border/subtle.
- **Caption** — body/sm, content/muted, above/below.

VARIANTS / PROPERTIES

- **Style:** bordered · borderless · zebra (alt rows on surface/subtle)
- **Cell align:** start · center · end (numeric → end)
- **Row state:** default · hover · selected
- Density flows from the frame's Context mode like everything else.

BINDINGS

| Part | Property | Token |
|--------------------|-------------|---------------------------------|
| Header text | font / fill | label/sm/* · content/secondary |
| Body text | font / fill | body/sm/* · content/primary |
| Header bg | fill | surface/raised |
| Row border | stroke | border/subtle |
| Cell padding-x / y | padding | table/cell-padding-x / -y (new) |
| Zebra row | fill | surface/subtle |

Figure + figcaption

COMPONENT

BUILD

| Part | Token |
|---------------------|--|
| Media slot | fill surface/subtle placeholder · radius radii/8 |
| Caption font / fill | body/sm/* · content/muted |
| Media → caption gap | space-8 |

Variants: caption position (below · overlay), align (start · center).

Divider

<hr>

COMPONENT

BUILD

DIVIDER/*

| Part | Property | Token |
|------------------|-------------|---|
| Line | stroke | border/subtle |
| Line | thickness | border-width/1 |
| Block margin | spacing | divider/margin-block (new) → space-16/24/32 |
| Label (optional) | font / fill | overline · content/muted |

Variants: orientation (horizontal · vertical), with-label (none · centred).

6 · Inline / text-level — marks

INLINE MARK

Mostly character styles or simple span treatments — a weight/style swap or a colour/background. Fast to define; easy to overlook precisely because they're small.

| Element | Build as | Treatment / token |
|---|------------|---|
| <code></code>
bold | char style | Asta Sans ships a Bold weight — bind it directly. Add a <code>font-style.bold</code> = "Bold" string token (only Regular/Medium/SemiBold exist today) so the binding is on-system. |
| <code></code> italic | char style | Confirm whether Asta Sans ships an italic style. If yes, bind it; if not, use a synthetic slant or a non-italic emphasis convention. |
| <code><mark></code>
highlight | span | Background tint — needs a highlight token (warning/brand tint at low chroma). NEW |
| <code></code> | char style | Strikethrough + <code>content/muted</code> |
| <code><ins></code> | char style | Underline + <code>content/primary</code> |
| <code><sub></code> / <code><sup></code> | char style | ~0.75em + baseline shift |
| <code><abbr></code> | char style | Dotted underline, <code>content/secondary</code> |
| <code><small></code> | text style | <code>body/xs</code> · <code>content/muted</code> |
| <code><q></code> inline quote | span | Curly quotes around <code>content/secondary</code> |
| <code><kbd></code> | component | Boxed key: monospace (§8) on <code>surface/raised</code> , <code>border/default</code> , <code>radii/4</code> , <code>space-4</code> pad |

7 · New token namespaces to create

Follow the established `dropdown/*` precedent: a component-specific namespace **inside the Context collection**, each value **aliasing an existing `space-*` primitive**, set per density mode. Process: *figma-variable-architecture* SKILL → "Adding a new framed-control property". Proposed alias values below (tune to taste before creating).

| New token | Dense | Compact | Comfort. | Spacious | Derivation |
|-----------------------------------|-----------------------|-----------------------|-----------------------|-----------------------|---|
| <code>list/item-gap</code> | <code>space-2</code> | <code>space-4</code> | <code>space-8</code> | <code>space-12</code> | Tracks <code>framed-control/gap</code> growth |
| <code>list/marker-gap</code> | <code>space-4</code> | <code>space-8</code> | <code>space-8</code> | <code>space-12</code> | Marker → content breathing room |
| <code>list/indent</code> | <code>space-16</code> | <code>space-24</code> | <code>space-28</code> | <code>space-32</code> | ~2× <code>padding-inline/md</code> |
| <code>quote/padding-inline</code> | <code>space-12</code> | <code>space-16</code> | <code>space-20</code> | <code>space-24</code> | Indent of the quote body |
| <code>code/padding</code> | <code>space-8</code> | <code>space-12</code> | <code>space-16</code> | <code>space-16</code> | Pre block inner padding |
| <code>table/cell-padding-x</code> | <code>space-8</code> | <code>space-12</code> | <code>space-16</code> | <code>space-20</code> | = <code>framed-control/md/padding-inline</code> |
| <code>table/cell-padding-y</code> | <code>space-4</code> | <code>space-8</code> | <code>space-12</code> | <code>space-12</code> | ~½ the x value |
| <code>divider/margin-block</code> | <code>space-12</code> | <code>space-16</code> | <code>space-24</code> | <code>space-32</code> | Vertical rhythm around rules |

Rule of thumb for any element with no token:

1. **Type** → reuse `body/*` or `heading/*` (never a new font scale).
2. **Colour** → reuse `content/*` · `surface/*` · `border/*` · `action/*`.
3. **Spacing** → if one `space-*` primitive fits, bind it directly. If it must **scale with density**, add a `<element>/*` namespace in Context, one alias per mode (the `dropdown/*` pattern).
4. Only mint a brand-new **primitive** if no scale value fits — for typographic web elements, essentially never (the exception: monospace, §8).

8 · The monospace gap

The system has exactly two faces — **Khand** (condensed display/label) and **Asta Sans** (body). **Neither is monospace.** Every code-bearing element (`inline code` , `code block` , `<kbd>` , `<samp>` , `<var>`) needs a fixed-width face. This is a real decision to make *before* building those components:

- **Add a `font-family.mono` primitive** — pick a face (e.g. a libre mono), add the variable to `Primitives` , back it up via the sync plugin, and bind code nodes to it. **Recommended** — keeps code elements on-system.
- **Defer code components** until the mono face is chosen, and build the non-code elements first.

Bold & italic: Asta Sans ships a **Bold** weight, so `<strong>` binds on-system — just add a `font-style.bold` = "Bold" string token (the weight primitive `font-weight.bold` = 700 already exists; only the style strings stop at SemiBold). **Italic** (`<em>`) is unconfirmed — check whether the family ships an italic style and bind it if so; otherwise use a synthetic slant or a non-italic emphasis convention. Document the choice in each component's description.

9 · Master checklist — one glance

| # | Element | Build | Status | New tokens? |
|----|---------------------|------------|--------|-------------------|
| 1 | Display / hero | Text style | EXISTS | — |
| 2 | Headings h1–h6 | Text style | EXISTS | — |
| 3 | Paragraph / body | Text style | EXISTS | — |
| 4 | Lead paragraph | Text style | ADD | — |
| 5 | Caption / helper | Text style | ADD | — |
| 6 | Overline / eyebrow | Text style | EXISTS | — |
| 7 | Address | Text style | ADD | — |
| 8 | Link | Component | BUILD | visited? (decide) |
| 9 | List + List Item | Component | BUILD | list/* |
| 10 | Description list | Component | BUILD | — |
| 11 | Blockquote | Component | BUILD | quote/* |
| 12 | Pull quote | Component | BUILD | — |
| 13 | Inline code | Component | BUILD | mono (§8) |
| 14 | Code block | Component | BUILD | code/* · mono |
| 15 | Table | Component | BUILD | table/* |
| 16 | Figure + figcaption | Component | BUILD | — |
| 17 | Divider / hr | Component | BUILD | divider/* |
| 18 | kbd | Component | BUILD | mono (§8) |
| 19 | strong | Char style | BUILD | — |
| 20 | em | Char style | BUILD | italic? (decide) |
| 21 | mark / highlight | Span | BUILD | highlight tint |
| 22 | del | Char style | BUILD | — |
| 23 | ins | Char style | BUILD | — |
| 24 | sub / sup | Char style | BUILD | — |
| 25 | abbr | Char style | BUILD | — |
| 26 | small | Text style | BUILD | — |
| 27 | q (inline quote) | Span | BUILD | — |

Suggested order: **(a)** decide the three open questions — monospace face, bold/italic strategy, link-visited token; **(b)** add the missing text styles + char styles (cheap); **(c)** build Link → List → Description list → Blockquote → Table → Code → the rest; **(d)** create the `list/* · quote/* · code/* · table/* · divider/*` namespaces as each component needs them; **(e)** write each component's description (the mandatory last step). Remember: inside every component, bind the four type fields inline — never a text style.